Class template deduction guides for "diamond operators"

Document: P0560R0
Date: 2017-02-01
Project: Programming Language C++
Audience: Library Evolution Working Group

Author: Miro Knejp (miro.knejp@gmail.com)

Abstract

This paper proposes a simplification of the syntax required to use library function types of the form `less<>()` (the so called "diamond operators") to `less()` by introducing class template deduction guides.

Proposal

The introduction of class template deduction allows for the simplification of the rather awkward syntax for the so called "diamond operators", such as `less<>()` or `plus<>()`. They all have in common a single template parameter which defaults to `void`. The simplification is achieved by creating deduction guides for the default constructors of these function types which deduce the template argument to `void`:

```
func() -> func<void>;
```

Proposed Wording

These changes are based on [N4618].

Change 20.14 [function.objects] paragraph 2 as follows:

```
// 20.14.5, arithmetic operations
template <class T = void> struct plus;
template <class T = void> struct minus;
template <class T = void> struct multiplies;
template <class T = void> struct divides;
template <class T = void> struct modulus;
template <class T = void> struct negate;
template <> struct plus<void>;
template <> struct minus<void>;
template <> struct multiplies<void>;
template <> struct divides<void>;
template <> struct modulus<void>;
template <> struct negate<void>;
```

```
plus()       -> plus<void>;
minus()      -> minus<void>;
multiplies() -> multiplies<void>;
divides()    -> divides<void>;
modulus()    -> modulus<void>;
negate()     -> negate<void>;
```

```
// 20.14.6, comparisons
template <class T = void> struct equal_to;
template <class T = void> struct not_equal_to;
template <class T = void> struct greater;
template <class T = void> struct less;
template <class T = void> struct greater_equal;
template <class T = void> struct less_equal;
template <> struct equal_to<void>;
template <> struct not_equal_to<void>;
template <> struct greater<void>;
template <> struct less<void>;
template <> struct greater_equal<void>;
template <> struct less_equal<void>;
```

```
equal_to()      -> equal_to<void>;
not_equal_to()  -> not_equal_to<void>;
greater()       -> greater<void>;
```

```
less()          -> less<void>;
greater_equal() -> greater_equal<void>;
less_equal()    -> less_equal<void>;
```

```
// 20.14.7, logical operations
template <class T = void> struct logical_and;
template <class T = void> struct logical_or;
template <class T = void> struct logical_not;
template <> struct logical_and<void>;
template <> struct logical_or<void>;
template <> struct logical_not<void>;
```

```
logical_and() -> logical_and<void>;
logical_or()  -> logical_or<void>;
logical_not() -> logical_not<void>;
```

```
// 20.14.8, bitwise operations
template <class T = void> struct bit_and;
template <class T = void> struct bit_or;
template <class T = void> struct bit_xor;
template <class T = void> struct bit_not;
template <> struct bit_and<void>;
template <> struct bit_or<void>;
template <> struct bit_xor<void>;
template <> struct bit_not<void>;
```

```
bit_and() -> bit_and<void>;
bit_or()  -> bit_or<void>;
bit_xor() -> bit_xor<void>;
bit_not() -> bit_not<void>;
```

References

[N4618] Working Draft, Standard for Programming Language C++

Acknowledgements

Adi Shavit and Simon Brand for initiating the thought process leading up to this paper.